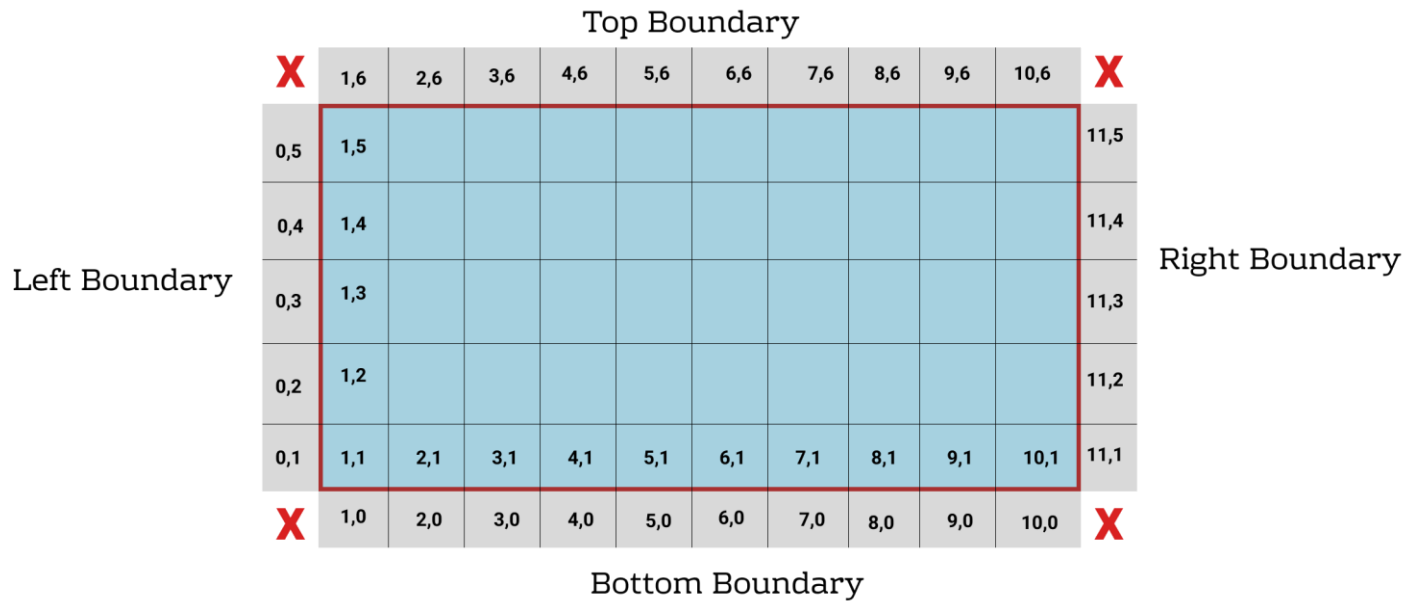


ChemE 6110 Mathematical Methods Final Project

Ameh Emmanuel Sunday esa62 Net ID



1.BOUNDARY CONDITIONS

- 1) The figure above shows a pictorial implementation of the boundary conditions for mesh $n_x=10$ $n_y=5$ using the ghost cells (In ash color) for a discretized domain of $L_y=1$ unit and $L_x=2$ units where L_y and L_x are the lengths of the domain in the y and x axis respectively. The cells in blue are the physical cells whose temperature using the Gauss Sidel method will be solved for. The points marked **X** are not included in the ghost cells because they are not used in calculating any temperature cell values in the physical domain. The red line at the edges of the box represents the boundary where the boundary conditions are applied.

I implemented the boundary conditions as follows:

Left Boundary Condition: $T=0$ (Dirichlet Boundary Condition)

$$\frac{T[1,j] + T[0,j]}{2} = 0 \text{ and hence } T[0,j] = -T[1,j]$$

Right Boundary Condition: $T=0$ (Dirichlet Boundary Condition)

$$\frac{T[nx+1,j]+T[nx,j]}{2} = 0 \text{ where } nx=10 \text{ and hence } T[nx+1,j] = -T[nx,j] \text{ or } T[11,j] = -T[10,j]$$

Bottom Boundary Condition: $T_y=0$ (Neumann Boundary Condition)

$$T[i, 0] = T[i, 1]$$

Top Boundary Condition: $T = \sin\left(\frac{\pi x}{2}\right)$ (Dirichlet Boundary Condition)

$$\frac{T[i,ny+1]+T[nx,j]}{2} = \sin\left(\frac{\pi x}{2}\right) \text{ which is same as}$$

$$T[i, ny + 1] = 2 \times \sin\left(\frac{\pi x}{2}\right) - T[i, ny] \text{ where } x = (i - 0.5) \times \Delta x.$$

All the cells were filled with an initial value of 0.0 at after the boundary conditions were defined to be updated at each Gauss Sidel iteration being done. The boundary conditions were also initialized at the cell centers of each ghost cell due to the finite volume numerical method being implemented. The grid being used is uniform as $\Delta x = \Delta y$ where $\Delta x = \frac{Lx}{nx}$ and $\Delta y = \frac{Ly}{ny}$.

Checking the ghost cells by hand: The ghost cells initially are everywhere except for the top boundary where we have the Dirichlet sine boundary condition. The general form of the ghost cells at the top boundary is $T[i, ny + 1]$ where $ny = 5$ and since the inner physical cells directly below the ghost are $T[i, ny]$ and are all 0 we can confirm by hand that the ghost cell values will be given as:

$$T[i, ny + 1] = 2 \times \sin\left(\frac{\pi x}{2}\right) \text{ where } x = (i - 0.5) \times \Delta x$$

$$\text{For cell 1 where } i=1, T[1,6] = 2 \times \sin\left(\frac{\pi \times 0.5 \times 0.2}{2}\right) = 0.3128689$$

$$\text{For cell 2 where } i=2, T[2,6] = 2 \times \sin\left(\frac{\pi \times 1.5 \times 0.2}{2}\right) = 0.90798099$$

$$\text{For cell 3, where } i=3, T[3,6] = 2 \times \sin\left(\frac{\pi \times 2.5 \times 0.2}{2}\right) = 1.4142135$$

$$\text{For cell 4, where } i=4, T[4,6] = 2 \times \sin\left(\frac{\pi \times 3.5 \times 0.2}{2}\right) = 1.782013$$

$$\text{For cell 5, where } i=5, T[5,6] = 2 \times \sin\left(\frac{\pi \times 4.5 \times 0.2}{2}\right) = 1.97537668$$

$$\text{For cell 6, where } i=6, T[6,6] = 2 \times \sin\left(\frac{\pi \times 5.5 \times 0.2}{2}\right) = 1.97537668$$

For cell 7, where $i=7$ $T[7,6] = 2 \times \sin\left(\frac{\pi \times 6.5 \times 0.2}{2}\right) = 1.7820130$

For cell 8, where $i=8$ $T[8,6] = 2 \times \sin\left(\frac{\pi \times 7.5 \times 0.2}{2}\right) = 1.4142135$

For cell 9, where $i=9$ $T[9,6] = 2 \times \sin\left(\frac{\pi \times 8.5 \times 0.2}{2}\right) = 0.90798099$

For cell 10, where $i=10$ $T[10,6] = 2 \times \sin\left(\frac{\pi \times 9.5 \times 0.2}{2}\right) = 0.31286893$

Cell Numbers $T[0,6]$ and $T[12,6]$ which are the corner cells of the top boundary are not checked as they are not used to calculate the values of any physical cell.

These values correspond to the code output of arrays for the top boundary :

The values of the Top Dirihlet boundary are:

0.31286893008046174

0.9079809994790936

1.4142135623730951

1.7820130483767356

1.9753766811902755

1.9753766811902753

1.7820130483767358

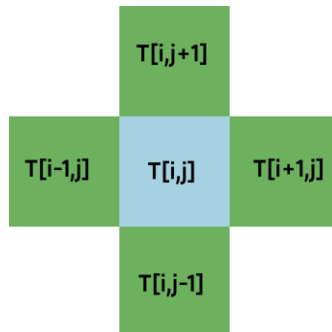
1.4142135623730951

0.9079809994790929

0.31286893008046196

2.POINT GAUSS-SEIDEL ITERATION SCHEME

The point Gauss-Sidel iteration scheme given that a uniform mesh is implemented is iterated numerous times for each physical cell until the steady state value is reached which is determined by the L_2 norm $||\delta T_{ij}||_2$ of the update of less than 10^{-7} . The formula for the Gauss Sidel Iterative method is given as:



$$T[i, j]^{k+1} = \frac{1}{4} [T[i-1, j]^{k+1} + T[i+1, j]^k + T[i, j-1]^{k+1} + T[i, j+1]^k]$$

Where $k + 1$ is the Value at the next time step and k is the value at the current time step. The information used to calculate the values of the temperatures of each physical cell comes from the boundary conditions. The number of iterations to get convergence for a mesh of $n_x = 10$ and $n_y = 5$ was 140 iterations.

The L_2 norm of the solution is 9.3846×10^{-8} which is calculated as the square of the difference between successive iterations and given by the formula:

$\sqrt{\frac{\sum (T[i, j]^{k+1} - T[i, j]^k)^2}{n_x \cdot n_y}}$ where T^{k+1} is the temperature at the next time step for each cell and T^k is the temperature value at the previous time step and the L2 norm error is given as the difference between the exact values and numerical solution calculated in the code.

The **L2 norm error** is given by the formula and calculated after convergence:

$$\sqrt{\frac{\sum (T_{\text{numerical}} - T_{\text{exact}})^2}{n_x \cdot n_y}} = 0.003392229 \text{ for a mesh of } n_x = 10 \text{ and } n_y = 5$$

The exact solution was calculated by defining a function and using the formula given as:

$T(x, y) = \frac{\sin(\pi x/2) \cosh(\pi y/2)}{\cosh(\pi/2)}$. To ensure that the values of the exact solution are calculated for the cell centers we have:

$$x = (i - 0.5)dx$$

$$y = (j - 0.5)dy$$

To prove that the L2 norm error is accurate I printed the values of the temperature for the numerical solution after convergence and the exact solution after convergence for the cell centers and they are equal with very little errors.

Visual inspection can be done to confirm similar temperature values after convergence in the picture on the next page for mesh $n_x, n_y = 10, 5$.

The **L2 norm error** of the Mesh 10 x 5 was **0.003392229611**.

```

The values of the temperature at cell centers calculated numerically:
[[ 0.          -0.06307772 -0.06925232 -0.08220587 -0.10320633 -0.13430938
  0.          ]
 [ 0.06307772  0.06307772  0.06925232  0.08220587  0.10320633  0.13430938
  0.17855955]
 [ 0.18305872  0.18305872  0.20097809  0.23857074  0.29951646  0.38978099
  0.51820001]
 [ 0.28512072  0.28512072  0.31303077  0.37158268  0.46650785  0.60709814
  0.80711542]
 [ 0.35927315  0.35927315  0.39444186  0.46822155  0.58783423  0.7649883
  1.01702475]
 [ 0.39825746  0.39825746  0.43724225  0.51902765  0.6516193  0.84799607
  1.12738061]
 [ 0.39825756  0.39825756  0.43724233  0.51902771  0.65161933  0.84799608
  1.1273806 ]
 [ 0.35927342  0.35927342  0.39444208  0.46822172  0.58783432  0.76498833
  1.01702472]
 [ 0.28512107  0.28512107  0.31303106  0.3715829  0.46650799  0.60709818
  0.80711538]
 [ 0.18305903  0.18305903  0.20097835  0.23857093  0.29951658  0.38978103
  0.51819997]
 [ 0.06307786  0.06307786  0.06925243  0.08220595  0.10320639  0.13430939
  0.17855954]
 [ 0.          -0.06307786 -0.06925243 -0.08220595 -0.10320639 -0.13430939
  0.          ]]

The values of the temperature at cell centers calculated with the exact solution:
[[0.          0.          0.          0.          0.          0.
  0.          ]
 [0.          0.06311563 0.06939629 0.08258261 0.10398677 0.1357387
  0.          ]
 [0.          0.18316868 0.20139588 0.23966408 0.30178136 0.39392904
  0.          ]
 [0.          0.28529191 0.31368144 0.37328555 0.47003548 0.61355886
  0.          ]
 [0.          0.35948877 0.39526167 0.47036723 0.59227926 0.77312927
  0.          ]
 [0.          0.39849638 0.43815094 0.5214061 0.65654662 0.8570204
  0.          ]
 [0.          0.39849638 0.43815094 0.5214061 0.65654662 0.8570204
  0.          ]
 [0.          0.35948877 0.39526167 0.47036723 0.59227926 0.77312927
  0.          ]
 [0.          0.28529191 0.31368144 0.37328555 0.47003548 0.61355886
  0.          ]
 [0.          0.18316868 0.20139588 0.23966408 0.30178136 0.39392904
  0.          ]
 [0.          0.06311563 0.06939629 0.08258261 0.10398677 0.1357387
  0.          ]
 [0.          0.          0.          0.          0.          0.
  0.          ]]

```

3.ACCURACY

The numerical and exact solutions were calculated for different mesh sizes which have 10×5 , 20×10 , 40×20 , and 80×40 control volumes. The tolerance values were reduced below 10^{-7} for each mesh to ensure that the error I measured was discretization error, not lack of convergence. We obtained different numbers of iterations to get to convergence for our different meshes.

Mesh (n_x, n_y)	Tolerance Value	Iterations for Convergence
10 x 5	10^{-7}	140
20 x 10	10^{-10}	773
40 x 20	10^{-12}	3555
80 x 20	10^{-10}	10116

As indicated and expected the lower the tolerance value and the finer the mesh, we get higher values of iterations to get to the convergence value at steady state. The L2 norm and L2 norm errors the meshes were also calculated and given in the table below:

Mesh (n_x, n_y)	Grid Spacings (Δx)	L ₂ norm Values	L ₂ norm error Values
10 x 5	2/10=0.2	9.8×10^{-8}	0.0033922296
20 x 10	2/20=0.1	9.78×10^{-11}	0.0008832988
40 x 20	2/40=0.05	9.96×10^{-13}	0.0002230965
80 x 20	2/80=0.025	9.99×10^{-11}	0.00005940856

To confirm the order of accuracy of our solution, we know that the L2 norm error $\sim (\Delta x)^2$

$M1_{error} \rightarrow (\Delta x)$ and $M2_{error} \rightarrow (\frac{\Delta x}{2})^2$ where M1 and M2 are two different mesh sizes.

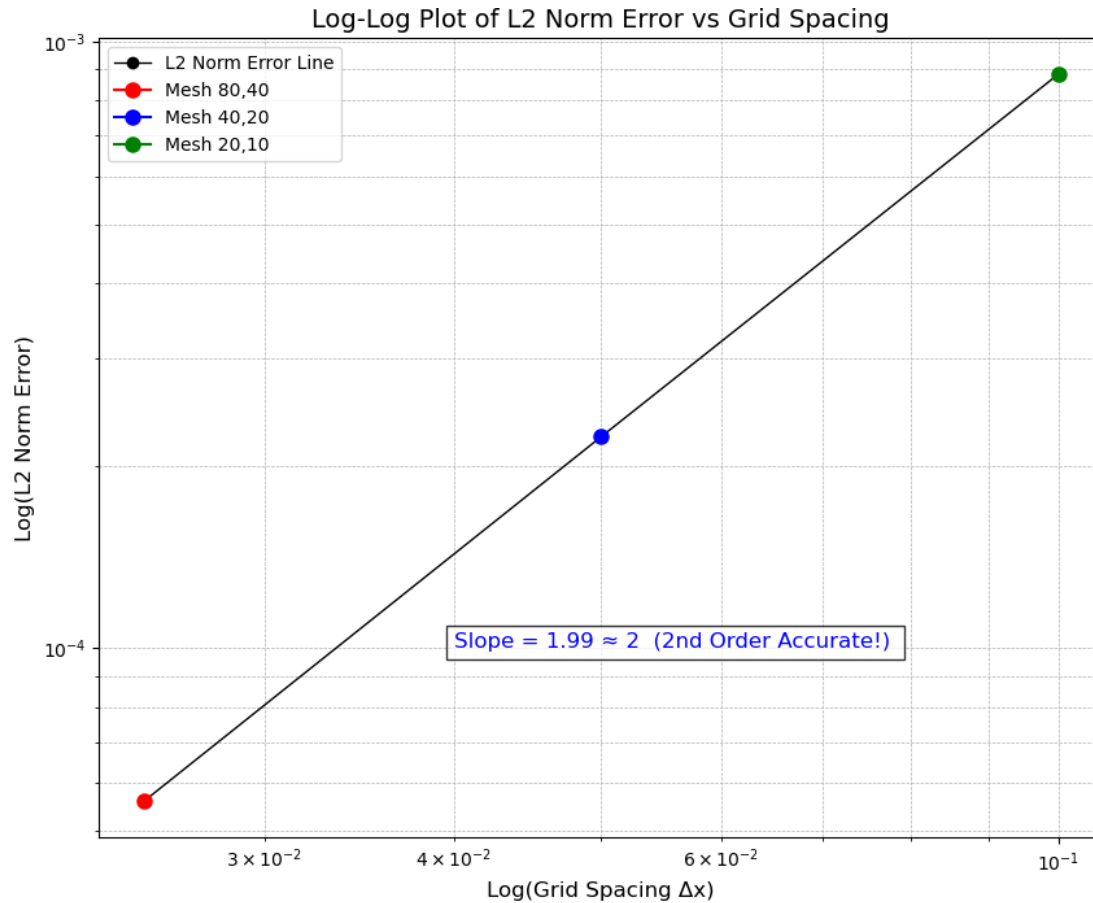
$$\frac{M1_{error}}{M2_{error}} = \frac{\Delta x}{\frac{\Delta x^2}{4}}, \text{ hence } M1_{error} = 4 \times M2_{error}.$$

$$\frac{Merror(10 \times 5)}{Merror(20 \times 10)} = \frac{0.0033922296}{0.0008832988} = 3.84 \sim 4$$

$$\frac{Merror(20 \times 10)}{Merror(40 \times 20)} = \frac{0.0008832988}{0.0002230965} = 3.95 \sim 4$$

$$\frac{Merror(40 \times 20)}{Merror(80 \times 40)} = \frac{0.0002230965}{0.00005940856} = 3.75 \sim 4$$

We observe that the finer the mesh the closer the value approaches exactly 4. To verify the second order accuracy of discretization a log-log plot of L2 norm error and Grid Spacing are plotted against each other, the slope which gives the order of accuracy = 2 indicating confirming that we have a 2nd order discretization scheme.



4.OVER-RELAXATION

With over relaxation the goal is to accelerate the convergence of the systems of linear equations. It introduces a relaxation factor ω which modifies the update of the solution in each iteration and overall reduces the number of iterations it takes to reach convergence. Over-relaxation amplifies the update step by scaling it with the relaxation factor $\omega > 1$, effectively "pushing" the solution further along its convergence path, thereby reducing the number of iterations required to meet a convergence criterion.

With Successive over relaxation, the value of the value of the temperature at the next time step is given as:

$T[i, j]^{k+1} = T[i, j]^k + \omega \delta T[i, j]^{k+1}$ where ω is a multiplier larger than 1 to get the over relaxation effect where:

$T[i, j]^{k+1}$ = This is the update value of $T[i, j]$ at iteration $k + 1$

$T[i, j]^k$ = This is the old value of $T[i, j]$ from iteration k

$\Delta T[i, j]$ = This is the change or correction in the value of $T[i, j]$ based on the neighbors. In Gauss-Seidel, this would be the difference between the newly computed average of neighbors and the old value.

ω = The over-relaxation factor, a multiplier greater than 1 to speed up convergence.

The formula for Gauss-Sidel with over- relaxation factor can also be written as:

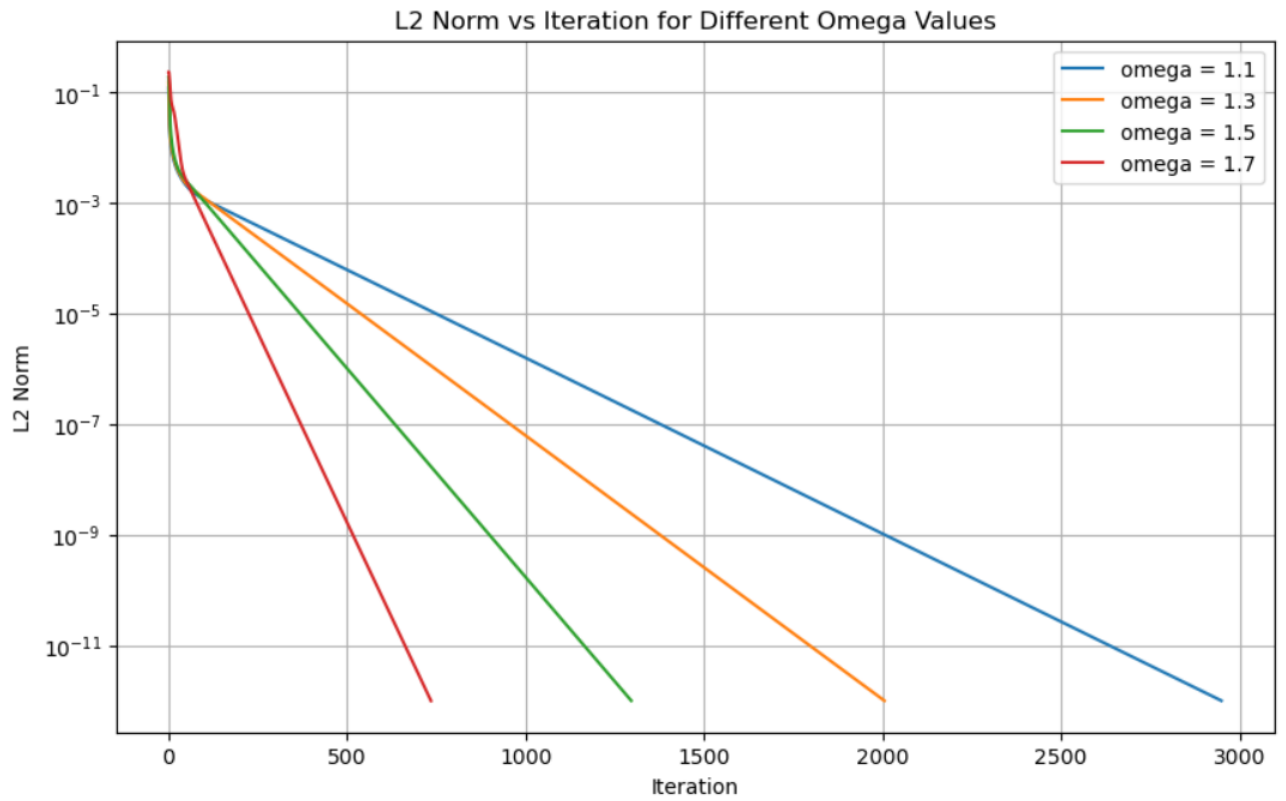
$$T[i, j]^{k+1} = (1 - \omega) \cdot T[i, j]^k + \omega \cdot \frac{1}{4} (T(i + 1, j)^k + T(i - 1, j)^{k+1} + T(i, j + 1)^k + T(i, j - 1)^{k+1})$$

Implementing this for a mesh of $nx = 40$ and $ny = 20$ with different values of $\omega = 1.1, 1.3, 1.5, 1.7$.

Over- relaxation factor ω	Iterations for Convergence
1.1	2950
1.3	2006
1.5	1297
1.7	736

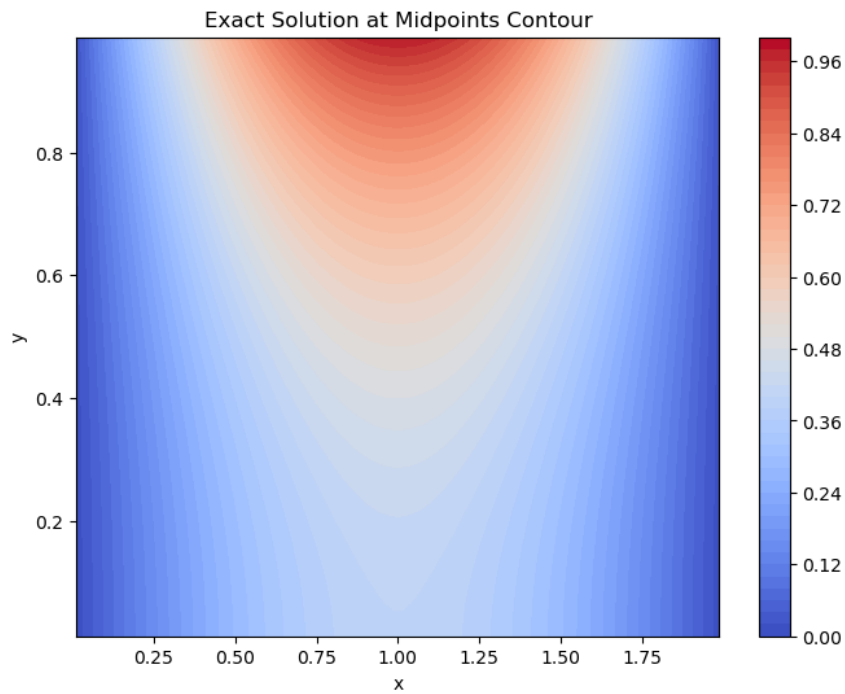
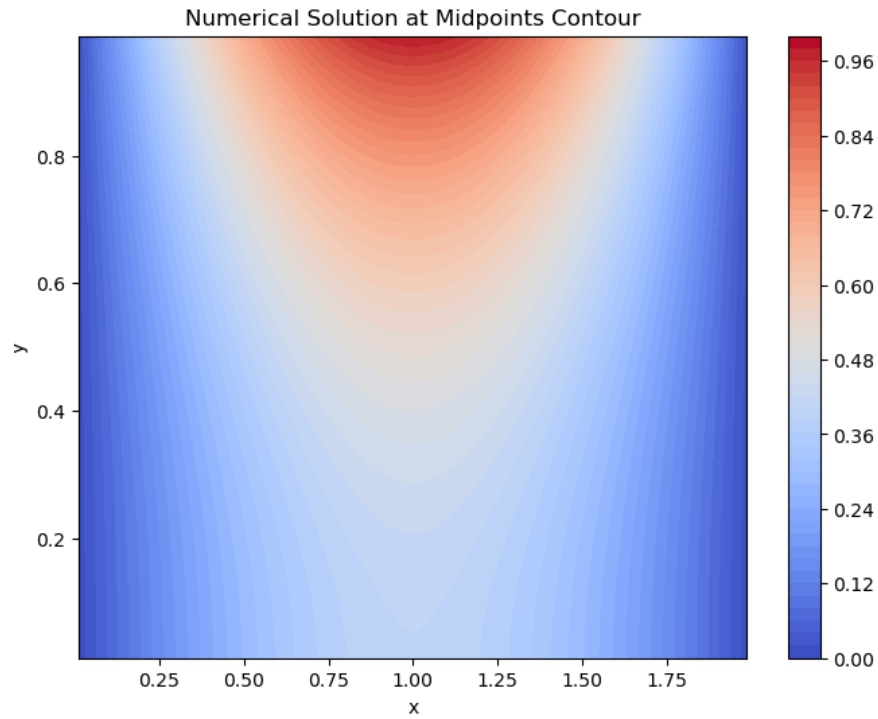
As expected, the number of iterations reduce as we increase the relaxation factor. The graph below shows the L_2 -norm of the solution update at each iteration as a function of

iteration count.



5.SOLUTION

2D contours of the temperature in the domain for the mesh with 80×40 control volume.



The plots above depict the numerical and exact solutions for the 80 x 40 mesh.

6.NOTES

- All code is written in python language and can be accessed in the attached python scripts with Vs code or jupyter notebook

- The Numerical and Exact Solutions for all the Meshes are Shown in each python script.
- The final temperature Values after Convergence for the Numerical and Exact Solutions are printed out for the purpose of comparison.